Windows Installation Guide for napari-swc-viewer

This guide installs `napari-swc-viewer` into a standard folder:

```
%USERPROFILE%\repos\napari-swc-viewer
```

`%USERPROFILE%` means your Windows home folder, usually something like:

```
C:\Users\yourname
```

The project uses Pixi to install Python, napari, and the plugin dependencies. You do not need to create a separate Python virtual environment.

1. Open PowerShell, Command Prompt, or Git Bash

PowerShell is recommended for this guide.

Open PowerShell

Use any one of these methods:

1. Click the Start menu, type `PowerShell`, and press `Enter`.
2. Right-click the Start button and choose `Terminal` or `Windows PowerShell`.
3. Press `Windows-R`, type `powershell`, and press `Enter`.

You do not usually need to run PowerShell as Administrator for this guide.

Open Command Prompt

Use any one of these methods:

1. Click the Start menu, type `cmd`, and press `Enter`.
2. Press `Windows-R`, type `cmd`, and press `Enter`.

Open Git Bash

Git Bash is installed by Git for Windows. If Git for Windows is installed:

1. Click the Start menu.
2. Type `Git Bash`.
3. Press `Enter`.

You can also right-click inside a folder in File Explorer and choose `Open Git Bash here` if that option is available.

2. Check Whether Git Is Installed

Open PowerShell and run:

```
git --version
```

If Git is installed, you should see a version number.

You can also check where Windows found Git:

```
where.exe git
```

If PowerShell says that `git` is not recognized, install Git for Windows.

3. Install Git for Windows

Official Git for Windows page:

https://gitforwindows.org/

Git installation reference:

https://git-scm.com/book/en/v2/Getting-Started-Installing-Git

Download and Run the Installer

1. Open a web browser.
2. Go to https://gitforwindows.org/.
3. Click `Download`.
4. Open the downloaded installer.
5. Accept the license and continue through the installer.

The default installer choices are mostly fine. Pay attention to the choices below.

Recommended Git for Windows Installer Choices

When the installer asks which components to install:

- Keep `Git Bash` enabled.
- Keep `Git GUI` enabled.
- Keep Windows Explorer integration enabled if you want right-click Git options.

When the installer asks for the default editor used by Git:

- Choose `Notepad` if you are not a developer.
- Choose `Visual Studio Code` only if VS Code is already installed and you are comfortable using it.
- Do not choose Vim unless you already know Vim.

When the installer asks how Git should be used from the command line:

- Choose `Git from the command line and also from 3rd-party software`.

When the installer asks about HTTPS transport:

- Use the default OpenSSL option.

When the installer asks about line endings:

- The default Windows line-ending option is fine.

When the installer asks about the terminal emulator for Git Bash:

- The default MinTTY option is fine.

When the installer asks about Git Credential Manager:

- Keep Git Credential Manager enabled. It helps Windows store GitHub sign-in credentials.

After installation finishes, close all open PowerShell, Command Prompt, and Git Bash windows. Open a new PowerShell window and run:

```
git --version
```

4. Do Not Use Vim For Git Commit Messages

Vim is a powerful text editor, but it is confusing if you have not used it before. Many non-developers get stuck when Git opens Vim for a commit message.

For non-developers, Notepad is strongly recommended.

Run this in PowerShell, Command Prompt, or Git Bash:

```
git config --global core.editor notepad
```

That tells Git to open Notepad when Git needs you to type a commit message.

If you are a developer and prefer Visual Studio Code, you may use this instead:

```
git config --global core.editor "code --wait"
```

Only use the VS Code option if the `code` command works from your terminal.

If You Are Already Stuck in Vim

If a terminal window shows many `~` characters down the left side and you cannot type normally, Git may have opened Vim.

To leave Vim without saving:

1. Press `Esc`.

2. Type:

```
:q!
```

3. Press `Enter`.

Then configure Notepad:

```
git config --global core.editor notepad
```

5. Create a `repos` Directory in Your Home Folder

PowerShell

Run:

```
New-Item -ItemType Directory -Force -Path "$env:USERPROFILE\repos"
Set-Location "$env:USERPROFILE\repos"
```

Confirm your location:

```
Get-Location
```

It should end with:

```
\repos
```

Command Prompt

Run:

```
mkdir "%USERPROFILE%\repos"
cd /d "%USERPROFILE%\repos"
```

If Command Prompt says the folder already exists, that is fine.

Git Bash

Run:

```
mkdir -p ~/repos
cd ~/repos
pwd
```

In Git Bash, `~/repos` is the same general location as `%USERPROFILE%\repos`.

6. Clone the GitHub Repository

The repository is public. You do not need special GitHub permission to download it.

1. Open a web browser.
2. Go to:

   https://github.com/nimh-dsst/napari-swc-viewer
3. Click the green `Code` button.
4. Choose either `HTTPS` or `SSH`.

Use `HTTPS` unless you already know that SSH keys are set up for your GitHub account.

Recommended: Clone with HTTPS

In PowerShell, make sure you are in `%USERPROFILE%\repos`, then run:

```
git clone https://github.com/nimh-dsst/napari-swc-viewer.git
```

Then enter the repository folder:

```
Set-Location "$env:USERPROFILE\repos\napari-swc-viewer"
```

Alternative: Clone with SSH

Only use SSH if your GitHub SSH keys are already configured.

```
git clone git@github.com:nimh-dsst/napari-swc-viewer.git
Set-Location "$env:USERPROFILE\repos\napari-swc-viewer"
```

Command Prompt Version

```
cd /d "%USERPROFILE%\repos"
git clone https://github.com/nimh-dsst/napari-swc-viewer.git
cd /d "%USERPROFILE%\repos\napari-swc-viewer"
```

Git Bash Version

```
cd ~/repos
git clone https://github.com/nimh-dsst/napari-swc-viewer.git
cd ~/repos/napari-swc-viewer
```

Confirm the Clone Worked

In PowerShell, run:

```
Get-ChildItem
```

You should see files such as:

```
README.md
MANUAL.MD
pixi.toml
src
docs
```

7. Install Pixi

Pixi is the dependency manager used by this project. It installs the right Python and napari environment for you.

Official Pixi installation page:

https://pixi.prefix.dev/latest/installation/

Open PowerShell and run:

```
powershell -ExecutionPolicy Bypass -c "irm -useb https://pixi.sh/install.ps1 | iex"
```

When the installer finishes, close all PowerShell, Command Prompt, and Git Bash windows. Open a new PowerShell window.

Then check Pixi:

```
pixi --version
```

If you see a Pixi version number, Pixi is installed.

If `pixi` Is Not Found

Close PowerShell and open it again, then retry:

```
pixi --version
```

If it still does not work, check whether this folder exists:

```
%USERPROFILE%\.pixi\bin
```

Pixi should add that folder to your `PATH`. Restarting Windows can also help Windows reload the updated `PATH`.

8. Launch napari with the Plugin

PowerShell

Run:

```
Set-Location "$env:USERPROFILE\repos\napari-swc-viewer"
pixi run napari
```

Command Prompt

Run:

```
cd /d "%USERPROFILE%\repos\napari-swc-viewer"
pixi run napari
```

Git Bash

Run:

```
cd ~/repos/napari-swc-viewer
pixi run napari
```

The first run can take several minutes because Pixi may need to download and install packages.

When napari opens, use the napari `Plugins` menu and open `Neuron Viewer`.

9. Put Launcher Scripts on the Desktop

The repository includes two Windows launcher examples:

```
run_scripts\run_napari.ps1
run_scripts\run_napari.bat
```

For non-developers, the BAT file is usually easiest because it can be double-clicked.

Copy the BAT Launcher to the Desktop

In PowerShell, run:

```
Copy-Item "$env:USERPROFILE\repos\napari-swc-viewer\run_scripts\run_napari.bat" "$env:U
SERPROFILE\Desktop\Run napari SWC Viewer.bat"
```

Double-click `Run napari SWC Viewer.bat` on your Desktop.

Copy the PowerShell Launcher to the Desktop

In PowerShell, run:

```
Copy-Item "$env:USERPROFILE\repos\napari-swc-viewer\run_scripts\run_napari.ps1" "$env:U
SERPROFILE\Desktop\Run napari SWC Viewer.ps1"
```

To run it from PowerShell:

```
powershell -ExecutionPolicy Bypass -File "$env:USERPROFILE\Desktop\Run napari SWC
Viewer.ps1"
```

If double-clicking the `.ps1` file opens it in Notepad instead of running it, use the BAT launcher or run the PowerShell command above.

10. Updating Later

To get the newest public version from GitHub later:

```
Set-Location "$env:USERPROFILE\repos\napari-swc-viewer"
git pull
pixi run napari
```

Troubleshooting

`git clone` Says the Folder Already Exists

If this command:

```
git clone https://github.com/nimh-dsst/napari-swc-viewer.git
```

says the folder already exists, the repository may already be downloaded. Try:

```
Set-Location "$env:USERPROFILE\repos\napari-swc-viewer"
git pull
```

SSH Clone Fails

If this command fails:

```
git clone git@github.com:nimh-dsst/napari-swc-viewer.git
```

use HTTPS instead:

```
git clone https://github.com/nimh-dsst/napari-swc-viewer.git
```

Pixi Takes a Long Time the First Time

That is expected. The first `pixi run napari` has to create the environment and download dependencies. Later launches should be faster.

The Desktop Launcher Cannot Find the Repository

The launchers assume this exact folder:

```
%USERPROFILE%\repos\napari-swc-viewer
```

If you cloned the repository somewhere else, either move it to that folder or edit the launcher script to use your actual path.

Reference Links

- Project repository: https://github.com/nimh-dsst/napari-swc-viewer
- Pixi installation: https://pixi.prefix.dev/latest/installation/
- Git for Windows: https://gitforwindows.org/
- Git installation guide: https://git-scm.com/book/en/v2/Getting-Started-Installing-Git
- GitHub cloning guide: https://docs.github.com/en/repositories/creating-and-managing-repositories/cloning-a-repository